CrossDesk — Devnet Integration Report

Porting an institutional Canton settlement desk from a local Daml 2.9.4 sandbox to the shared **HackCanton devnet node** (NODERS `hackcanton-01`, Canton 3.x)

Status: LIVE ON-CHAIN. The desk is deployed on the shared node and **settles real transactions**. First atomic DvP (2026-07-19): *alice-crossdesk → bob-crossdesk · 10 cETH @ 3,200 USDC* — receipt visible only to Alice/Bob/Auditor (sub-transaction privacy on shared infrastructure). Also live: cETH/CBTC/USDC instruments, the LX1 in-kind basket (NAV 970 USDC, computed on-ledger; create + redeem round-tripped), and a 2-of-3 committee that struck an official NavFixing. Public hosted demo: **https://crossdesk-devnet-app.web.app** (Firebase + Cloud Run, auto-refreshing OIDC token).

1 · What CrossDesk is

An institutional settlement desk on Canton: **atomic DvP** (two tokenised legs move together or not at all — no principal/Herstatt risk), a **sealed Market-on-Close** call auction (order book invisible until it clears — no front-running), a **K-of-N NAV committee** (an official closing price no single venue can print), and an **in-kind ETF/basket builder** (create/redeem fund shares against underlyings, atomically). Assets: **cETH** (onRails) and **CBTC** (BitSafe) as first-class instruments.

2 · Deployment model — who does what

Canton separates *writing code* from *running a node*. We wrote the code; NODERS runs the node. The three admin steps require operator rights on the participant, which only the node runner holds.

Step	Who	What
Build Daml → **DAR**	us	`daml build` packages every template into one `.dar`
Upload the DAR	**node operator**	admin-only — makes the CrossDesk templates exist on the node
Allocate parties	**node operator**	admin-only — creates on-ledger identities
Grant `actAs` / `readAs`	**node operator**	admin-only — attaches permissions to our login
Connect backend + drive desk	us	our JWT + TLS over the Ledger API

3 · Auth & connection model

```
appsfactory login (email + password)
     ↓  → NODERS identity provider (Keycloak)
   JWT issued  — the "keycard": proves you're sborjas, scope daml_ledger_api
     ↓  sent inside TLS (encrypted tunnel; node presents a CA-signed cert)
   Participant node — Ledger API (gRPC :443)
     ↓  node checks JWT, then readAs/actAs rights per party
   Synchronizer — orders & finalizes the transaction across participants
```

JWT = identity (a bearer token, no client cert). **TLS** (not mTLS) = the secure pipe; only the server shows a cert, the client authenticates with the JWT. **readAs vs actAs** — two rights the node attaches per party: *readAs* = view (read-only), *actAs* = submit transactions as that party (create, exercise, settle). Reads need only readAs; **every state-changing action needs actAs**. The appsfactory account is specific to this node (NODERS' gatekeeper) — the pattern (an IdP issuing JWTs) is universal.

4 · Why two backend builds

The shared node runs Canton 3.x, which rejects the SDK 2.9.4 package format and speaks Ledger API v2. To avoid breaking the working local demo, the port lives in a copy (`backend-devnet/`); the original `backend/` still runs locally, unchanged.

	backend/ (original)	backend-devnet/ (port)
Target	local sandbox	HackCanton shared node
Daml SDK / LF	2.9.4 / LF 1.14	3.4.11 build / LF 2.2
Ledger API	v1	v2
Party management	admin gRPC service	config roster (v2 dropped the admin service)
Status	untouched, still runs	ported, compiles clean, connected

5 · The Ledger API v1 → v2 port (code)

All changes are confined to two files (`LedgerConnection.java` , `LedgerService.java`) plus a config property and the run script.

v1 (2.9.4)	v2 (3.x)	Why
PartyManagement admin channel	read a configured `ledger.parties` roster	v2 rxjava bindings dropped the admin service; shared-node parties are fixed
`getActiveContractSetClient()`	`getStateClient()`	renamed in v2
`getActiveContracts(f, p, v)`	+ `ledgerEndOffset`	v2 ACS queries are offset-anchored
`CommandsSubmission.create(app,id,cmds)`	+ `Optional<synchronizerId>`	v2 added an optional synchronizer selector
`getLedgerId()` handshake	removed — a clean `connect()` confirms	v2 has no ledger-id handshake
`submitAndWaitForTransactionTree` → `TransactionTree`	`submitAndWaitForTransaction(sub, LEDGER_EFFECTS)` → `Transaction`	Canton 3.3+ removed the tree command endpoints; `LEDGER_EFFECTS` carries the same node set (`CreatedEvent` implements both

	interfaces, so decode is unchanged)

6 · Problems hit, and the fix for each

1. **DAR rejected — "expected LF 2.1/2.2/2.3, got 1.14."** Node is Canton 3.x. → Rebuilt with SDK 3.4.11 (LF 2.2).
2. **LF 2.x removed contract keys.** `Instrument` had a key. → Removed it (desk queries the ACS, never by key); Kiryl uploaded it (package `72ec9833…`).
3. **Backend wouldn't compile against v2 (182 errors).** → Traced to two files and ported the API surface (§5). Clean compile + 41 MB jar.
4. **`javap` showed the wrong API.** It hit the 2.9.4 jar first. → Re-ran against the exact 3.4.0 jars for the real v2 signatures.
5. **Port 8080 in use.** Local build owns it. → Devnet build defaults to 8090; both run side by side.
6. **`UNIMPLEMENTED: SubmitAndWaitForTransactionTree`.** Node doesn't serve the removed endpoint. → Switched to `submitAndWaitForTransaction` + `LEDGER_EFFECTS`. Error gone; command now reaches the node.
7. **`PERMISSION_DENIED` on every submit — solved via the participant's Grafana/Loki logs.** Querying the node's logs for the denied `tid` revealed two real causes: (a) in Ledger API v2 the submission's `applicationId` **IS the userId** — it must equal the token's `sub` (set `LEDGER_APPLICATION_ID`); (b) actAs claims match **full party ids** (`…::1220…`), not labels — resolve labels before submitting. → **HTTP 201: real contracts created.**

7 · Result — live on-chain (2026-07-19)

All proven against the live shared node:

- ✅ **Daml package deployed (LF 2.2, `72ec9833…`); backend connects over TLS + JWT.**
- ✅ **Reads AND writes end-to-end: instruments (cETH/CBTC/USDC) created, holdings issued.**
- ✅ **Atomic DvP settled: *alice-crossdesk → bob-crossdesk · 10 cETH @ 3,200 USDC* — receipt `006ef8c599…`, visible only to Alice/Bob/Auditor (sub-transaction privacy on a shared node).**
- ✅ **In-kind ETF lifecycle: LX1 basket (0.10 cETH + 0.01 CBTC/share) — create 10 shares at on-ledger NAV 970.00 USDC, redeem round-trip.**
- ✅ **K-of-N governance: 2-of-3 committee proposed, attested and finalised an official NavFixing.**
- ✅ **Sealed MOC auction: orders rest sealed; the venue clears the book at the attested close.**

> **Try it:** the hosted demo at **https://crossdesk-devnet-app.web.app** serves this live on-chain state (Firebase Hosting + Cloud Run, OIDC token auto-refresh — stays connected unattended). Thanks to the NODERS team for the node, party allocation and actAs grants.

CrossDesk · HackCanton S2 · github.com/superbigroach/canton-dvp-settlement-desk · Full technical narrative in `DEVNET_INTEGRATION.md`; reproduce via `backend-devnet/RUNBOOK.md`.